

エージェントの故障に対応可能な マルチ UAV システムの構築

滑川研究室

D2 村上卓弥

June 6, 2025

Abstract

This study focuses on resilient formation control for leader-follower multi-agent systems. While earlier methods demonstrated effectiveness through theoretical analysis and simulations, they were limited to first-order dynamics and has not considered distributed failure detection and collision avoidance. The current research includes hardware-based validation using actual drones and updates to the control laws for implementation, as well as fundamental investigations aimed at theoretical advancements that address these limitations.

1 はじめに

本研究では、マルチエージェントシステムを対象に、エージェントの故障に対応可能なレジリエントなマルチエージェントシステムの実現を目指している。2025 年度においては、2024 年度研究において提案した故障対応制御則に関して、実機を用いた検証を行うための開発と制御則の更新に加え、理論的な発展に向けた基礎検討を行った。

2 研究の概要

2.1 背景

近年、UAV (Unmanned Aerial Vehicle) や UGV (Unmanned Ground Vehicle) といった移動ロボットの実用化が著しく進展しており、物流、データ収集、農業など幅広い分野での活用が加速化している。このような流れの中、複数の自律ロボットが協調して全体の効率を向上させるマルチエージェントシステムの制御への関心は高く、従前より活発な研究が行われてきた [1]。しかしながら、システム内のロボット数が増加するに従い、一部のロボットに故障等の不具合が発生する可能性も高まるが、マルチエージェントシステムにおける故障に着目した研究は、マルチエージェントの制御そのものに関する研究と比較して少ないという現状がある。

マルチエージェントシステムの故障対応に関する従来研究では、故障によるシステムへの影響を最小限に抑えるためのロバスト制御器の設計に主眼が置かれてきた [2–6]。しかし、これらの研究の多くは、システムが補償可能な故障を前提としており、補償能力を超える故障が発生した場合、その影響がシステム全体に拡大し、全体の機能を損なうおそれがある。

マルチエージェントシステムの制御においては、フォーメーション制御や合意制御が重要なテーマであり、その中では速度や位置といった状態変数を所望の値に収束させることが求められる。その中でもリーダー・フォロワー構造は、システムの柔軟性、効率性、スケーラビリティといった利点を有するため、広く用いられている [7–10]。リーダーフォロワー構造におけるエージェントの異常に対してレジリエントなフォーメーション制御の実現を目的とした研究も存在するが [11, 12]、多くはリーダーの異常を想定していない。さらに、リーダー交代に関する研

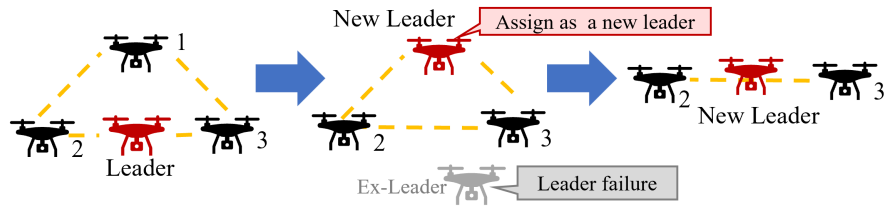


Fig. 1: Concept of the proposed approach

究 [13] は、フォロワーの異常を十分に考慮しておらず、フォロワーの異常が生じた場合、システムのデッドロックが生じるリスクがある。

これらを踏まえ、2024 年度に実施した研究 (以下、2024 年度研究) [14, 15] では、リーダーフォロワー構造のマルチエージェントシステムにおけるフォーメーション制御を対象とし、補償不可能な故障に対応可能なレジリエントな制御手法の検討および、リーダーの故障に対応するためのアルゴリズムの構築という 2 点の課題に取り組み、Fig. 1 に示すような、リーダーを含むいずれかのエージェントに補償困難な故障が発生した場合でも、フォーメーションを維持可能なマルチエージェントシステムを設計することを目的とし、理論的解析及びシミュレーションを通じて有効性を確認した。ただし、2024 年度研究では、単純な一次系で検討を行っており、ドローンへの直接的な適用に課題がある点や、故障検知がエージェント毎に行われており、エージェント自身の故障検知が機能しない場合を想定してない、故障エージェントとの衝突回避が考慮されていないという課題があった。

これらの課題を踏まえ、現時点での研究の主眼としては、2024 年度研究での提案制御則の実機実証に向けた取組に加え、分散的なエージェントの故障検知の実現の 2 点を中心に検討を行っている。

本資料の構成は以下のとおりである。第 3 章では、実機実験に向けた取組として、制御入力に対する機体の応答を確認した機体動作特性の取得、解析を行いつつ、複数機制御プログラムのアップデート状況について報告する。第 4 章では第 3 章の結果を踏まえた制御則の修正を行い、第 5 章において理論的發展に向けた検討について紹介する。

3 実機実験

実機実験に向けた準備等の進捗について報告する。主に複数制御のシステム構成の構築を行った。主要な進捗をまとめると以下の通りである。

- ドローン (TELLO-EDU) への制御入力に対する機体の応答に関するデータを取得、応答特性の解析を行った
- キーボード操作によりドローン 2 機を同時制御するコードの開発を行い、2 機のリアルタイム制御に成功した

まず、現状のシステム構成を Fig. 2 に示す。主要な進捗を赤字で示している通り、現時点では、実機の制御関係の開発としては、オペレータが 2 機をリアルタイム同時制御するというところまで達成した。加えて次に、制御則への落とし込みを想定した機体特性の取得について説明する。

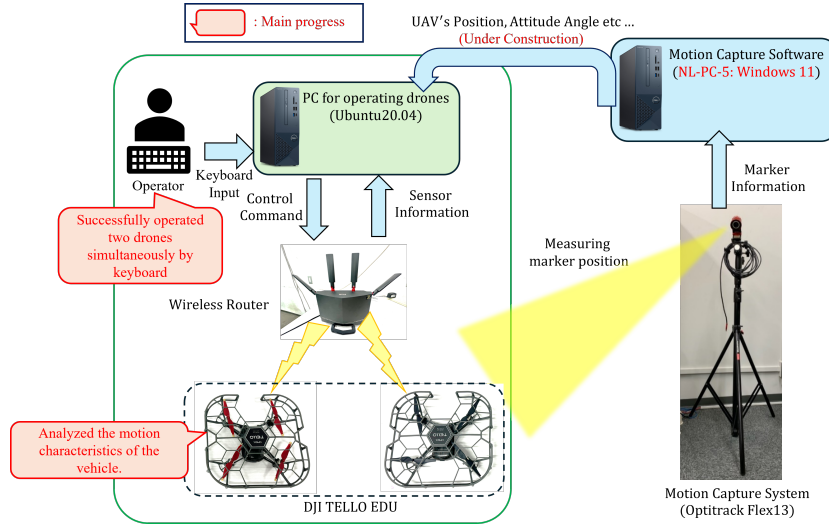


Fig. 2: Configure of the experimental system

3.1 機体特性の取得

TELLO-EDU には、飛行中に操作指令を印加しない際に機体に備わっている下方センサによりその場にとどまるホバリング機能、指令量に応じて機体を一定角度傾斜させ、その角度を保持する角度保持制御が備わっている。機体特性の取得のため、実際にドローンを飛行させ、一定時間ホバリングを行った後、左右方向に最大傾斜指示を行った。まず、実験の設定及び MOCAP 上の座標軸を Fig. 3 に示す。ドローンの 4 隅に球形の再帰性反射マーカを取り付け、ドローンをモーションキャプチャエリア内で飛行させ、左右方向に複数回往復する動作を行った。図に示している通り、座標系が右手系で x 軸: 前後方向, z 軸: 左右方向, y 軸: 上下方向となっていることに留意されたい。

次に、離陸からホバリング、左右への最大傾斜指示 (100: 0-100 の範囲で指定可能) を複数回実施した際のドローンの傾斜角度、位置、速度のモーションキャプチャの時系列データを Fig. 4 に示す。ただし、利用しているモーションキャプチャから直接的に取得できる情報は傾斜角度、位置であり、速度は位置の時間差分をもとに別途算出したものである。これらの時系列データから、ドローンの離陸後、全期間にわたって安定的にデータ取得できていることが確認できた。

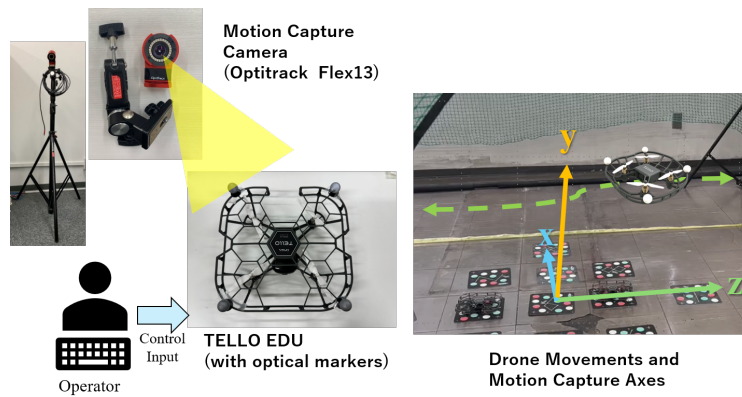


Fig. 3: Overview of experimental setup and motion capture axes

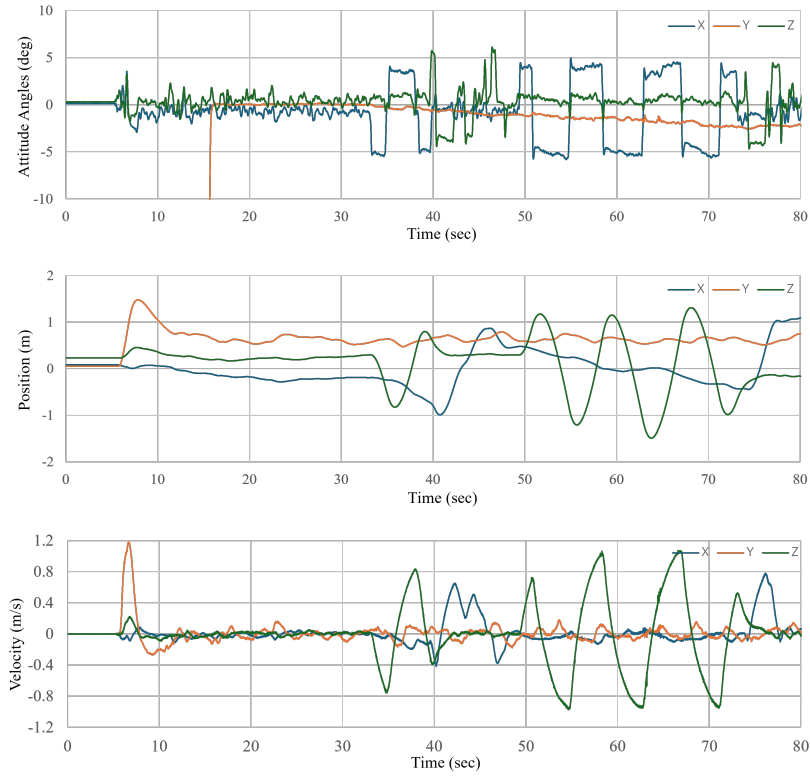


Fig. 4: Motion capture data of drone in flight

次に、ドローンの制御入力への応答に着目するため、Fig. 4 から左右一往復分 (32 秒～40 秒) を切り出して拡大したものを Fig. 5 に示す。まず、傾斜角について、制御入力に応じて傾斜角が概ね 5 度で維持されていることがわかった。次に速度について、傾斜角に応じて速度が概ね線形で増減しており、概ね等加速度運動となっていることが確認できた。この結果により、制御入力に応じてドローンの傾斜角を一定に保つための角度保持制御が働いていること、その結果として加速度入りに近似した形でドローンが高度を維持したまま水平面上を移動することが確認できた。なお、Fig. 5 の 36 秒から 38 秒付近では加速度が減少しているが、この主要因として空気抵抗等の影響が考えられる。ただし今回の機体特性取得にあたっては、実験室内の防護ネットの端から端までをフルレートで制御したような際に顕在化する事象であると考えられ、想定している実験の規模感では無視できるものとし、今後の加速度に関する解析においては入力直後から 1 秒間の速度傾斜の値を機体加速度を算出することにした、

ここまでで説明した最大傾斜角指示 (100) のパターンに加え、指示量 50, 25, 0 (=ホバリング) の 3 パターンについても同様の飛行試験によるデータ取得を行った。その際の傾斜角度のデータを Fig. 6、加速度のデータを Fig. 7 に出力した。Fig. 6 より、制御入力に応じて概ね線形に傾斜角度が増大していることが確認できる。次に Fig. 7 より、制御入力に対して機体の加速度が概ね線形に観測されていることがわかる。

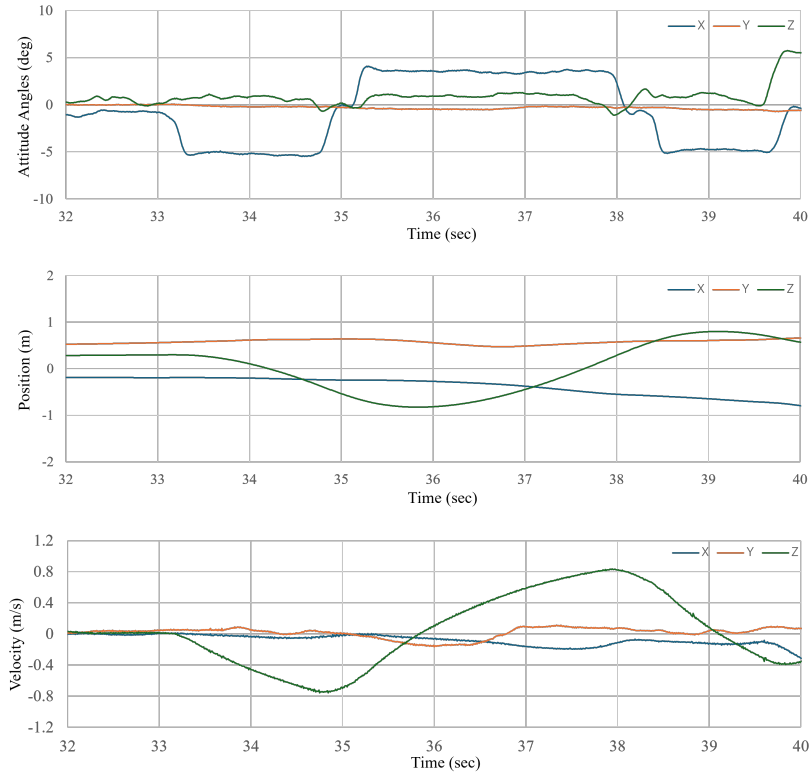


Fig. 5: Motion capture data of drone in flight (zoom-in)

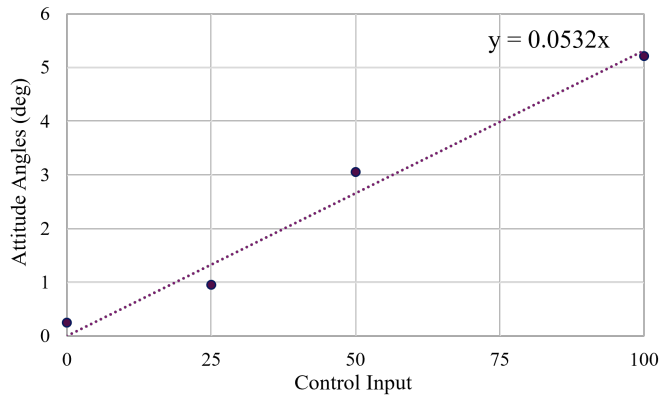


Fig. 6: Relationship between control input and attitude angles

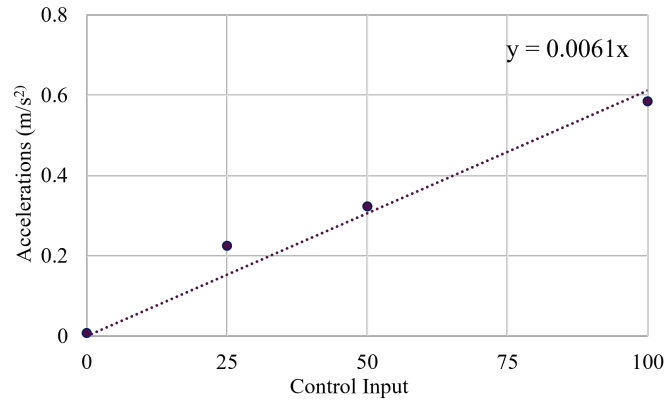


Fig. 7: Relationship between control input and accelerations

この結果により，TELLO-EDU においては機体の特性が模擬的に加速度入力の二次系として扱えることが確認できた．更に，線形近似を用いて機体への加速度入力 u に対して，オペレータが入力する u_{cmd} の間には， $u = 0.006 \cdot u_{cmd}$ ， $u_{cmd} \in [0, 100] \subset \mathbb{R}$ という関係性が成立することが確認できた．

次に，2 機をリアルタイム制御した際の様子を Fig. 8 に示す (動画も上映する)．離陸後，ホバリングし，左右前後方向に移動していることがわかる ()．

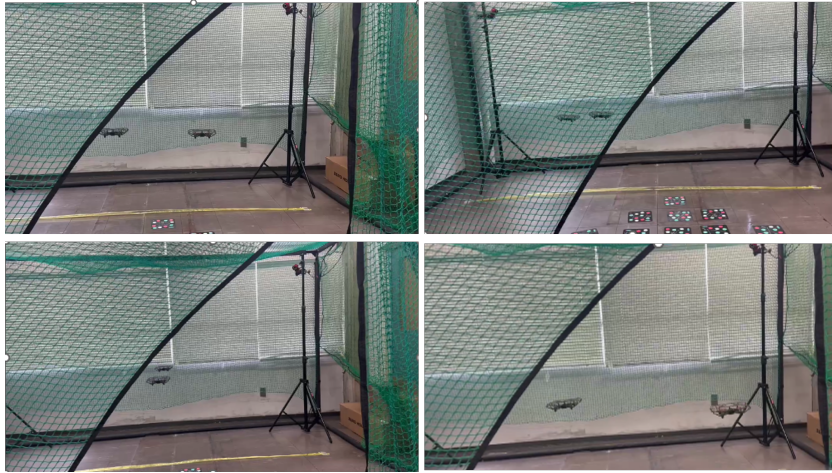


Fig. 8: Scene of multiple drone flights, Top-left: Hovering; Top-right: Movement to the Left; Bottom-left: Backward Movement; Bottom-right: Movement to the Right

現在，MOCAP PC からドローンの操作に用いる Ubuntu PC へのモーションキャプチャデータのストリーミングを確立するための開発作業を実施中である．MOCAP からのデータストリーミングが確立できれば 2024 年度研究の実機実証のためのシステム構成，データのやり取り等の基盤的な課題は全て解決できたこととなり，制御則自体の移植・実装作業に集中することができることとなるため，着実に開発作業を継続していきたいと考えている．

4 実機実験のための制御則

4.1 想定するモデルの整理

前項の結果を踏まえて、実機実験で想定する Quad-rotor の運動モデルについて説明する。Quad-rotor の運動モデルは本質的には各軸の運動が他の軸の運動に影響を及ぼす複雑かつ非線形なモデルである。これに対し、近年では様々な近似・線形化モデルが提案されている。例えば、一定の仮定のもとで水平方向の運動を 4 次システム、鉛直方向の運動を 2 次システムで表現することで制御の簡略化を図った事例 [18] などがある。その他の研究動向としては、Quad-rotor の姿勢および位置の運動を SE(3) の群構造上で厳密に表現するモデルも事例が存在する。

しかし、実際にはドローンに搭載されるフライトコントローラの性能向上や制御アルゴリズムの発達により、各軸の運動が実質的に独立した線形システムとして扱える場合が増加している。特に、水平面上の運動を加速度入力 of 線形 2 次システムとみなした Quad-rotor モデルを用いた協調搬送システムを構築し、実機検証を行った事例 [10] も存在している。

これらの背景と、本研究の実機実験で利用する Tello-Edu でも前項で検証した通り、水平面上の移動を加速度入力の線形 2 次システムとみなして扱えることが確認できたことから、本研究においては以降、各軸を独立した加速度入力の線形 2 次システムとして Quad-rotor の運動モデルを扱うこととする。

ここから、線形 2 次システムとした制御則について説明する。同一の N 機のエージェントで構成され三次元平面上を動作するマルチエージェントシステムとし、各エージェントは次式で表される連続時間線形 2 次システムを考える。

$$\begin{cases} \dot{r}_i(t) = v_i(t) \\ \dot{v}_i(t) = u_i(t), \quad (i = 1, 2, \dots, N) \end{cases} \quad (1)$$

ここで、 $r_i(t) \in \mathbb{R}^2, u_i(t) \in \mathbb{R}^2$ は、それぞれ i 番目のエージェントの位置及び制御入力を表す。

4.2 実機用フォーメーション制御則

本項では、複数のドローンがフォーメーションを達成するための水平面上の制御則の概要について説明する。

本研究においては、上述の制御目的達成のため、合意アルゴリズムをフォーメーション制御に適用した手法 [9] を用いる。本手法では、リーダーの動作にフォロワーがフォーメーション形状を維持したまま追従する、リーダー・フォロワー構造の制御を行う。次に、後述する理論的解析のため、以下の仮定を置く。

仮定 1. 全ての非故障フォロワー機は故障発生後もリーダー機から連結 (*connected*) しているものとする (ただしリーダーに直接接続する必要はない)。

仮定 2. ネットワークのグラフ構造は無向グラフであるものとする。

リーダーが 1 個、フォロワーが $N - 1$ 個の計 N 個のエージェントで構成されるシステムのエージェント i に対する水平面上のフォーメーション制御則は、以下の式で表される。

$$u_i = - \sum_{j=1}^{N+1} w_{ij} \left[k_1(r_i - r_j) - l_i k_1(d_i - d_j) + k_2(\dot{r}_i - \dot{r}_j) - l_i k_2(\dot{d}_i - \dot{d}_j) \right] \quad (2)$$

ここで、 $r_i(t) \in \mathbb{R}^2, v_i(t) \in \mathbb{R}^2, u_i(t) \in \mathbb{R}^2$ はそれぞれマルチエージェントシステム N の中のエージェント i の位置、速度、および加速度入力を表す。リーダーは任意のエージェント $L = \{1, \dots, N\}$ から選ばれ、添え字 $N + 1$ はリーダーに目標位置を送信するオペレータを示し、 r_{N+1} はオペレータから送信される目標位置を示している。次に、 l_i は次式のように表される。

$$l_i = \begin{cases} 0 & \text{for } i \text{ is a leader} \\ 1 & \text{otherwise} \end{cases} \quad (3)$$

また, $w_{ij}(t) \in \mathcal{A}$ は, エージェント間の接続状態を表す時変の隣接行列 $\mathcal{A} \in \mathbb{R}^{(N+1) \times (N+1)}$ の要素であり, 次式で表現される.

$$w_{ij}(t) = \begin{cases} a_{ij}\eta_i\eta_j & \text{for } i \text{ connected from } j \\ 0 & \text{otherwise} \end{cases} \quad (4)$$

ここで, $a_{ij} \in \mathcal{A}$ は, 当初のエージェントの接続状態を表現している. $\eta_i(t)$ は各エージェントの信頼度を表す値であり, 完全に信頼できるエージェントは 1, 信頼できないエージェントは 0 として表現される. この導出方法については後述する.

リーダーは自身が移動しながら, 直接接続されているフォロワーに対して, リーダー位置からみたすべてのエージェントの目標相対位置 $d_i(t) \in \mathbb{R}^2$ を決定し, それがネットワーク構造を通じて, 全フォロワーにその情報が伝わる. このネットワーク構造を用いて, リーダーはリーダーの位置 $r_L(t) \in \mathbb{R}^2$ から見た各エージェントの目標相対位置 d_i を送信し, 各フォロワーは隣接フォロワーの状況も加味して自身の制御入力決定する. 本制御則に対する定理と証明は後述する.

4.3 故障検出手法

本項では故障検出手法について説明する. 移動ロボットにおいてはさまざまな種類の故障が想定されるが, 本研究では発生する故障に関し, 制御入力に正しくない値に置き換わる, いわゆるアクチュエータ故障を想定する. アクチュエータ故障は以下の 4 つのタイプが存在する [16]. それぞれを式で表現すると次式のように示される.

$$u_i^f(t) = \begin{cases} f_{\max}, & \text{(a) Hard-over} \\ f u_i(t), & \text{(b) Loss of effectiveness} \\ u_i(t_f), & \text{(c) Lock-in-place} \\ [0 \ 0]^\top, & \text{(d) Float} \end{cases} \quad (5)$$

ここで, $u_i^f(t) \in \mathbb{R}^2$ は故障発生中に実際にドローンに入力される値であり, $f_{\max} \in \mathbb{R}^2$ はアクチュエータの最大入力, $f \in \mathbb{R}$ は減衰係数, $t_f \in \mathbb{R}$ は故障発生時からの時間経過を示す.

発生する故障に関して以下の仮定を置く.

仮定.3 発生する故障は 4 つのタイプのいずれかのアクチュエータ故障とし, 各エージェントのセンサ情報等の観測誤差はなく, 故障エージェントは非故障エージェント同様に隣接エージェントとの情報交換を継続する.

次に, 故障検知に関しては残差法 (Residual method) [17] を用いる. 残差信号は, 実際のシステムとオブザーバによって得られる理想的な状態との違いを表す. 残差信号は, 正常な状態では 0 に近い値を保つように設計されており, 故障等により実際の挙動とモデルの乖離が生じることにより 0 から大きく逸脱し, 故障検知が可能となる. さて, 式 (2) に基づいて, 分散オブザーバは次式のように表現できる.

$$\ddot{\hat{r}}_i = - \sum_{j=1}^{N+1} a_{ij} \left\{ k_1(\hat{r}_i - \hat{r}_j) - k_1(d_i - d_j) + k_2(\dot{\hat{r}}_i - \dot{\hat{r}}_j) - k_2(\dot{d}_i - \dot{d}_j) \right\} + m\epsilon_i \quad (6)$$

ここで, $\hat{r}(t) \in \mathbb{R}^2$ は推定値であり, $m \in \mathbb{R}$ はオブザーバゲインである. 残差 $\epsilon_i(t) \in \mathbb{R}^2$ は, エージェント i の観測値と推定値の偏差, およびエージェント i に隣接するエージェントの観測値と推定値の偏差の合計として定義さ

れる．残差 $\epsilon_i(t)$ は次式のように表される．

$$\epsilon_i(t) = (r_i - \hat{r}_i) + \sum_{j=1}^N w_{ij}(r_j - \hat{r}_j) \quad (7)$$

この方法により，フォーメーション制御のための分散オブザーバと，オブザーバへのフィードバックに使用される残差 ϵ_i の導出が可能となる．故障検出は，残差 ϵ_i を別途定義した閾値と比較することによって可能となる．ただし，入力値が故障等の影響によりモデルと乖離した場合，推定値と観測値の差異が増加することにより結果として大きな残差が生じる．しかし，実際には，突風などによる一時的な挙動の乱れや一時的な故障からの回復が想定される．したがって，故障検出に基づき直ちにエージェントとの情報を遮断することは望ましくない．これを考慮して，各エージェントの信頼度に基づいてエージェント間の接続の重みを決定するアルゴリズムを設計する．次に，各エージェントの信頼度を計算する方法について説明する．

各エージェントの信頼度を評価するための信頼度関数を導入する．まず残差が事前に設定した閾値を超えたとき，すなわち， $\|\epsilon_i\| > \tau$ となった際に故障検知が行われる．各エージェントは，故障検知の継続時間に応じて自身への“dissapointment”を増加させる．この“dissapointment”は次式のように表される．

$$\dot{\delta}_i = -\alpha\delta_i(t) + (1 - e^{-\beta t_i})\rho_i(t) \quad (8)$$

$$\rho_i(t) = \begin{cases} 1 & \|\epsilon_i\| > \tau \\ 0 & \text{otherwise} \end{cases} \quad (9)$$

ここで， $\alpha \in \mathbb{R}$ と $\beta \in \mathbb{R}$ は，それぞれ δ_i の減少と増加の速度を調整する正の定数であり， $t_i \in \mathbb{R}$ はエージェント i が故障を検出してからの経過時間を表す．エージェント i で故障が検出されると， ρ_i は 1 となり，それ以外の場合は 0 となる．エージェント i が故障を検出している間，第二項が増加し，これにより $\delta_i \in \mathbb{R}$ が増加する．エージェントが故障から回復すると，第一項が δ_i を減少させる．

さらに，信頼度 $\eta_i(t) \in \mathbb{R}$ を次式のように定義する．

$$\eta_i(t) = \max(1 - \delta_i, 0) \quad (10)$$

これにより，信頼度 η_i は 0 から 1 の範囲で表現されることになる．この η_i を制御に用い， η の低下したリーダーを交代することにより，故障エージェントからの影響局限，リーダーの交代等が可能になる．

4.4 実機用制御則の理論的解析

本稿では提案制御則の理論的解析を行う．まず，証明を行う定理は以下のとおりである．

定理 1. $N - 1$ 機のフォロワー ($N - 1 \geq 1$) と 1 機のリーダーから構成される N 機のエージェントからなるマルチエージェントシステムを考える．制御則 (2) を適用したシステム (1) は，仮定 1 から 3 が満たされており，制御ゲイン $k_1 > 0$ かつ $k_2 > 0$ を満たすよう制御ゲインを選択する時，故障有無に関わらず非故障エージェントによりフォーメーションが漸近的に達成される．

4.4.1 式の変形

まず目標相対位置に関して，リーダーフラグ l_i を組み込み次式のように表現する．

$$\tilde{h}_i(t) = r_i(t) - l_i d_i(t) \quad (11)$$

次に、速度に関しても同様の操作を行い、次式のように表現する．

$$\tilde{v}_i(t) = v_i(t) - l_i \dot{d}_i(t) \quad (12)$$

これらの変換により、制御則 (2) は以下のように書き換えられる．

$$\dot{\tilde{v}}_i = - \sum_{j=1}^{N+1} w_{ij} \left[k_1(\tilde{h}_i - \tilde{h}_j) + k_2(\tilde{v}_i - \tilde{v}_j) \right] \quad (13)$$

式 (13) は、変換された状態変数を用いた制御則の表現である．この形式で扱うことにより、リーダーとフォロワーを統一的に扱うことができるため、以降の証明が容易になる．

次に、全エージェントの状態を次元（位置、速度）ごとにまとめた状態ベクトルとして、

$$\tilde{h} = [\tilde{h}_1^\top, \tilde{h}_2^\top, \dots, \tilde{h}_{N+1}^\top]^\top \in \mathbb{R}^{2(N+1)} \quad (14)$$

$$\tilde{v} = [\tilde{v}_1^\top, \tilde{v}_2^\top, \dots, \tilde{v}_{N+1}^\top]^\top \in \mathbb{R}^{2(N+1)} \quad (15)$$

と定義する．時変隣接行列 $\mathcal{A}$ を次数行列 $\mathcal{D} \in \mathbb{R}^{(N+1) \times (N+1)}$ から引くことにより得られるグラフラプラシアン $\mathcal{L}$ を用いることで、システムは以下のように表現できる．

$$\begin{bmatrix} \dot{\tilde{h}} \\ \dot{\tilde{v}} \end{bmatrix} = \begin{bmatrix} 0 & I_{2(N+1)} \\ -k_1 \mathcal{L} \otimes I_2 & -k_2 \mathcal{L} \otimes I_2 \end{bmatrix} \begin{bmatrix} \tilde{h} \\ \tilde{v} \end{bmatrix} \quad (16)$$

ここで、 $\otimes$ はクロネッカー積を表し、 I_2 は 2 次元単位行列である．クロネッカー積を用いることで、二次元平面上の各エージェントの x 成分と y 成分を適切に扱うことができる．

状態ベクトル $\tilde{h}$ と $\tilde{v}$ を以下のように分割する．

$$\tilde{h} = [\tilde{h}_A^\top, \tilde{h}_{N+1}^\top]^\top \quad (17)$$

$$\tilde{v} = [\tilde{v}_A^\top, \tilde{v}_{N+1}^\top]^\top \quad (18)$$

ここで、 $\tilde{h}_A = [\tilde{h}_1^\top, \tilde{h}_2^\top, \dots, \tilde{h}_N^\top]^\top \in \mathbb{R}^{2N}$ 、 $\tilde{v}_A = [\tilde{v}_1^\top, \tilde{v}_2^\top, \dots, \tilde{v}_N^\top]^\top \in \mathbb{R}^{2N}$ である．

グラフラプラシアン $\mathcal{L}$ も同様に分割する．まず、グラフラプラシアン $\mathcal{L}$ の定義を確認しておく．グラフラプラシアン $\mathcal{L}$ は、次数行列 $\mathcal{D}$ から隣接行列 $\mathcal{A}$ を引くことで得られる．

$$\mathcal{L} = \mathcal{D} - \mathcal{A} \quad (19)$$

ここで、 $\mathcal{L}$ の対角要素 d_{ii} はエージェント i が接続しているエージェントの合計数を表し、それ以外の隣接行列 $\mathcal{A}$ は、ノード間の接続関係を表し、ノード i とノード j が接続している場合に $a_{ij} = 1$ 、そうでない場合に $a_{ij} = 0$ となる．

グラフラプラシアン $\mathcal{L}$ は、 $\mathcal{L}$ から $N+1$ 行、 $N+1$ 列を除去した $\mathcal{M} \in \mathbb{R}^{N \times N}$ を用いて次式のように分割できる．

$$\mathcal{L} = \begin{bmatrix} \mathcal{M} & \mathcal{L}_{N1} \\ \mathcal{L}_{1N} & \mathcal{L}_{11} \end{bmatrix} \quad (20)$$

ここで, $\mathcal{L}_{N1} \in \mathbb{R}^{N \times 1}$, $\mathcal{L}_{1N} \in \mathbb{R}^{1 \times N}$, $\mathcal{L}_{11} \in \mathbb{R}$ である.

グラフラプラシアン $\mathcal{L}$ の各行の和は 0 になることから, 任意の i について, $\sum_{j=1}^{N+1} \mathcal{L}_{ij} = 0$ が成り立つ. そのため, $\mathcal{L}_{1N}$ は $\mathcal{M}$ の各行の和の負値として次式のように表現できる.

$$\mathcal{L}_{1N} = -\mathcal{M}\mathbf{1}_N \quad (21)$$

ここで, $\mathbf{1}_N \in \mathbb{R}^N$ は全ての要素が 1 のベクトルである.

また, $\mathcal{L}_{N1} = \mathbf{0}_N^\top$ となる. ここで, $\mathbf{0}_N \in \mathbb{R}^N$ は全ての要素が 0 のベクトルである. このことから, $\mathcal{L}_{11} = 0$ となる.

以上より, グラフラプラシアン $\mathcal{L}$ は次式のように分割できる.

$$\mathcal{L} = \begin{bmatrix} \mathcal{M} & -\mathcal{M}\mathbf{1}_N \\ \mathbf{0}_N^\top & 0 \end{bmatrix} \quad (22)$$

ここから, この $\mathcal{M}$ を用いた式変形を行い安定性解析を行っていく. まず, 式 (22) を用いて, 式 (16) を書き換えると,

$$\begin{bmatrix} \dot{\tilde{h}}_A \\ \dot{\tilde{v}}_A \end{bmatrix} = \begin{bmatrix} 0 & I_{2N} \\ -k_1\mathcal{M} \otimes I_2 & -k_2\mathcal{M} \otimes I_2 \end{bmatrix} \begin{bmatrix} \tilde{h}_A \\ \tilde{v}_A \end{bmatrix} + \begin{bmatrix} 0 \\ k_1\mathcal{M}\mathbf{1}_N \otimes \tilde{h}_{N+1} + k_2\mathcal{M}\mathbf{1}_N \otimes \tilde{v}_{N+1} \end{bmatrix} \quad (23)$$

となる. ここで, $\mathbf{1}_N \otimes \tilde{h}_{N+1} \in \mathbb{R}^{2N}$ は $\tilde{h}_{N+1} \in \mathbb{R}^2$ を N 回繰り返したベクトル $\{\tilde{h}_{N+1}, \tilde{h}_{N+1}, \dots, \tilde{h}_{N+1}\}$ である.

式 (23) の右辺第 1 項はエージェントの動作に対応する項で微分方程式における斉次部分 (システムの内部ダイナミクスを記述する部分) に対応し, 第 2 項は目標位置を表現する項で非斉次部分 (外部入力を表す部分) に対応している. まず非斉次部分を 0 として, 斉次方程式としての安定性を考える.

4.4.2 斉次方程式の安定性解析

まず, 斉次方程式の部分について考える.

$$\begin{bmatrix} \dot{\tilde{h}}_A \\ \dot{\tilde{v}}_A \end{bmatrix} = \begin{bmatrix} 0 & I_{2N} \\ -k_1\mathcal{M} \otimes I_2 & -k_2\mathcal{M} \otimes I_2 \end{bmatrix} \begin{bmatrix} \tilde{h}_A \\ \tilde{v}_A \end{bmatrix} \quad (24)$$

行列 $\mathcal{M}$ の固有値を λ , 対応する固有ベクトルを ϕ とする.

$$\mathcal{M}\phi = \lambda\phi \quad (25)$$

ここで, 行列 $\mathcal{M}$ は正定行列であり, 仮定 1 と仮定 2 が満たされる限りその固有値はすべて正である (詳細は Appendix 参照).

次に, システム行列

$$A = \begin{bmatrix} 0 & I_{2N} \\ -k_1\mathcal{M} \otimes I_2 & -k_2\mathcal{M} \otimes I_2 \end{bmatrix} \quad (26)$$

の固有値を求める．この過程では， $\mathcal{M} \otimes I_2$ の固有値が重要となるため，クロネッカー積の固有値に関する性質に関する検討を行う．

まず，任意の行列 $C \in \mathbb{R}^{m \times m}$ と行列 $D \in \mathbb{R}^{n \times n}$ のクロネッカー積 $C \otimes D \in \mathbb{R}^{mn \times mn}$ は以下のように定義される．

$$C \otimes D = \begin{bmatrix} c_{11}D & c_{12}D & \cdots & c_{1m}D \\ c_{21}D & c_{22}D & \cdots & c_{2m}D \\ \vdots & \vdots & \ddots & \vdots \\ c_{m1}D & c_{m2}D & \cdots & c_{mm}D \end{bmatrix} \quad (27)$$

行列 C の固有値を $\psi_1, \psi_2, \dots, \psi_m$ ，対応する固有ベクトルを $\kappa_1, \kappa_2, \dots, \kappa_m$ とし，行列 D の固有値を $\omega_1, \omega_2, \dots, \omega_n$ ，対応する固有ベクトルを $\xi_1, \xi_2, \dots, \xi_n$ とする．このとき，クロネッカー積 $C \otimes D$ の固有値は $\psi_i \omega_j$ ($i = 1, 2, \dots, m$, $j = 1, 2, \dots, n$) となり，対応する固有ベクトルは $\kappa_i \otimes \xi_j$ となる．

証明は以下のように行うことができる． C と D の固有値と固有ベクトルの定義から．

$$C\kappa_i = \psi_i \kappa_i, \quad D\xi_j = \omega_j \xi_j \quad (28)$$

という関係が成り立つ．ここで，クロネッカー積 $C \otimes D$ の固有値と固有ベクトルの関係について，以下の式変形から得ることができる．

$$(C \otimes D)(\kappa_i \otimes \xi_j) = C\kappa_i \otimes D\xi_j \quad (29)$$

$$= \psi_i \kappa_i \otimes \omega_j \xi_j \quad (30)$$

$$= \psi_i \omega_j (\kappa_i \otimes \xi_j) \quad (31)$$

これにより， $C \otimes D$ の固有値は， C, D の固有値を組み合わせた $\psi_i \omega_j$ であり，対応する固有ベクトルも同様に $\kappa_i \otimes \xi_j$ であることがわかった．

本問題では， $C = \mathcal{M}, D = I_2$ とし， $\mathcal{M}$ の固有値を λ としている． I_2 は 2 次元単位行列であり，その固有値はすべて 1 である（重複度 2）．

したがって，先ほどのクロネッカー積の固有値に関する性質から， $\mathcal{M} \otimes I_2$ の固有値は重複度が 2 の $\lambda \cdot 1 = \lambda$ となる．この重複度は，物理的には二次元平面上のシステムを表しており， x 方向と y 方向の状態が同じダイナミクスを持つことを意味している．

4.4.3 システム行列 A の固有値解析

行列 A の固有値を μ とし，対応する固有ベクトルを $v = [v_1^\top, v_2^\top]^\top$ とすると，次式の関係が成り立つ．

$$Av = \mu v \quad (32)$$

これを展開すると，次式が得られる．

$$\begin{bmatrix} 0 & I_{2N} \\ -k_1 \mathcal{M} \otimes I_2 & -k_2 \mathcal{M} \otimes I_2 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \mu \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} \quad (33)$$

これにより，次式の連立方程式を得ることができる．

$$v_2 = \mu v_1 \quad (34)$$

$$-k_1(\mathcal{M} \otimes I_2)v_1 - k_2(\mathcal{M} \otimes I_2)v_2 = \mu v_2 \quad (35)$$

第一式 $v_2 = \mu v_1$ を第二式に代入すると、

$$-k_1(\mathcal{M} \otimes I_2)v_1 - k_2(\mathcal{M} \otimes I_2)(\mu v_1) = \mu^2 v_1 \quad (36)$$

$$-(k_1 + k_2\mu)(\mathcal{M} \otimes I_2)v_1 = \mu^2 v_1 \quad (37)$$

が得られる。ここで上式は、 k_1, k_2, μ が定数であることから、 A の固有ベクトル v_1 が $(\mathcal{M} \otimes I_2)$ の固有ベクトルの形をとっている。このため、 $(\mathcal{M} \otimes I_2)$ と A が共通の固有ベクトルを持つことが示された。そのため、 $(\mathcal{M} \otimes I_2)$ の固有値 λ に関して、 $(\mathcal{M} \otimes I_2)v_1 = \lambda v_1$ という関係が成り立つ。この関係を踏まえて式変形を行うと、

$$-(k_1 + k_2\mu)\lambda v_1 = \mu^2 v_1 \quad (38)$$

$$-k_1\lambda - k_2\lambda\mu = \mu^2 \quad (39)$$

$$\mu^2 + k_2\lambda\mu + k_1\lambda = 0 \quad (40)$$

という二次方程式が得られる。これがシステム行列 A の固有値 μ に関する特性方程式である。この特性方程式の解は、二次方程式の解の公式を用いて次式の形式で

$$\mu = \frac{-k_2\lambda \pm \sqrt{(k_2\lambda)^2 - 4k_1\lambda}}{2} \quad (41)$$

と表される。ここで、 $k_1 > 0$ 、 $k_2 > 0$ 、および $\lambda > 0$ であることに注意しつつ解の安定性を分析するために、判別式 $D = (k_2\lambda)^2 - 4k_1\lambda$ の符号によって場合分けを行って検討する。

場合 1: $D > 0$ (実数の異なる 2 つの解)

この場合、二次方程式は 2 つの異なる実数解を持つ。

$$\mu_1 = \frac{-k_2\lambda + \sqrt{D}}{2}, \quad \mu_2 = \frac{-k_2\lambda - \sqrt{D}}{2} \quad (42)$$

$k_2 > 0$ および $\lambda > 0$ より $-k_2\lambda < 0$ であり、 $\sqrt{D} < k_2\lambda$ であれば $\mu_1 < 0$ となる。この関係性を確認すると、

$$\sqrt{D} < k_2\lambda \quad (43)$$

$$\sqrt{(k_2\lambda)^2 - 4k_1\lambda} < k_2\lambda \quad (44)$$

$$(k_2\lambda)^2 - 4k_1\lambda < (k_2\lambda)^2 \quad (45)$$

$$-4k_1\lambda < 0 \quad (46)$$

となり、 $k_1 > 0$ および $\lambda > 0$ の定義よりこの関係が常に成り立つことが確認できた。したがって $\mu_1 < 0$ である。

また、 μ_2 については、 $-\sqrt{D} < 0$ より、明らかに $\mu_2 < 0$ である。

したがって、 $D > 0$ の場合、解 μ_1 と μ_2 は共に負の実数となる。

場合 2: $D = 0$ (重解)

この場合、二次方程式は重解となり、

$$\mu_1 = \mu_2 = -\frac{k_2\lambda}{2} \quad (47)$$

となる。 $k_2 > 0$ および $\lambda > 0$ より、解 μ_1 と μ_2 は共に負の実数となる。

場合 3: $D < 0$ (複素共役解)

この場合、二次方程式は複素共役の解を持つ。

$$\mu_{1,2} = -\frac{k_2\lambda}{2} \pm j\frac{\sqrt{|D|}}{2} \quad (48)$$

ここで j は虚数単位である。この解の実部は $-\frac{k_2\lambda}{2}$ であり、 $k_2 > 0$ および $\lambda > 0$ より負である。

以上の分析から、 $k_1 > 0$ および $k_2 > 0$ であれば、すべての場合においてシステム行列 A の固有値 μ の実部は負となる。したがって、斉次方程式の解は漸近安定である。

4.4.4 非斉次方程式の特殊解

前節では斉次方程式の安定性を証明したが、実際のシステムは非斉次方程式であり、外部入力項が存在する。非斉次方程式は以下のように表される。

$$\begin{bmatrix} \dot{\tilde{h}}_A \\ \dot{\tilde{v}}_A \end{bmatrix} = \begin{bmatrix} 0 & I_{2N} \\ -k_1\mathcal{M} \otimes I_2 & -k_2\mathcal{M} \otimes I_2 \end{bmatrix} \begin{bmatrix} \tilde{h}_A \\ \tilde{v}_A \end{bmatrix} + \begin{bmatrix} 0 \\ k_1\mathcal{M}\mathbf{1}_N \otimes \tilde{h}_{N+1} + k_2\mathcal{M}\mathbf{1}_N \otimes \tilde{v}_{N+1} \end{bmatrix} \quad (49)$$

ここで、右辺の第二項は外部入力項であり、リーダー（オペレータ）の目標位置 $\tilde{h}_{N+1}$ と目標速度 $\tilde{v}_{N+1}$ に依存している。

非斉次方程式の特殊解を求めるために、目標位置が最終的に一定値 $\tilde{h}_{N+1}^*$ に収束し、目標速度がゼロになる状態、つまり $\tilde{v}_{N+1} = 0$ となる場合を考える。

定常状態では $\dot{\tilde{h}}_A = 0$ および $\dot{\tilde{v}}_A = 0$ である。この条件を満たす特殊解を求める。目標位置が一定値 $\tilde{h}_{N+1}^*$ に収束し、目標速度が $\tilde{v}_{N+1} = 0$ となる場合、特殊解の候補として以下の形を考える。

$$\begin{bmatrix} \tilde{h}_A^* \\ \tilde{v}_A^* \end{bmatrix} = \begin{bmatrix} \mathbf{1}_N \otimes \tilde{h}_{N+1}^* \\ 0 \end{bmatrix} \quad (50)$$

ここで、 $\tilde{h}_A^* = \mathbf{1}_N \otimes \tilde{h}_{N+1}^*$ は、すべてのフォロワーの位置がリーダーの目標位置 $\tilde{h}_{N+1}^*$ と同じになることを意味し、 $\tilde{v}_A^* = 0$ はすべてのフォロワーの速度がゼロになることを意味する。

この特殊解の候補が実際に非斉次方程式を満たすかどうかを検証する。定常状態では $\dot{\tilde{h}}_A = 0$ および $\dot{\tilde{v}}_A = 0$ であるため、非斉次方程式は以下ようになる。

$$\begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 & I_{2N} \\ -k_1\mathcal{M} \otimes I_2 & -k_2\mathcal{M} \otimes I_2 \end{bmatrix} \begin{bmatrix} \mathbf{1}_N \otimes \tilde{h}_{N+1}^* \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ k_1\mathcal{M}\mathbf{1}_N \otimes \tilde{h}_{N+1}^* \end{bmatrix} \quad (51)$$

これを展開すると。

$$\begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \cdot (\mathbf{1}_N \otimes \tilde{h}_{N+1}^*) + I_{2N} \cdot 0 \\ -k_1(\mathcal{M} \otimes I_2)(\mathbf{1}_N \otimes \tilde{h}_{N+1}^*) - k_2(\mathcal{M} \otimes I_2) \cdot 0 + k_1 \mathcal{M} \mathbf{1}_N \otimes \tilde{h}_{N+1}^* \end{bmatrix} \quad (52)$$

第一成分は明らかに 0 である。第二成分については、

$$-k_1(\mathcal{M} \otimes I_2)(\mathbf{1}_N \otimes \tilde{h}_{N+1}^*) + k_1 \mathcal{M} \mathbf{1}_N \otimes \tilde{h}_{N+1}^* \quad (53)$$

ここで、クロネッカー積の性質を用いると、

$$(\mathcal{M} \otimes I_2)(\mathbf{1}_N \otimes \tilde{h}_{N+1}^*) = (\mathcal{M} \mathbf{1}_N) \otimes (I_2 \tilde{h}_{N+1}^*) \quad (54)$$

$$= \mathcal{M} \mathbf{1}_N \otimes \tilde{h}_{N+1}^* \quad (55)$$

したがって、第二成分は、

$$-k_1(\mathcal{M} \mathbf{1}_N \otimes \tilde{h}_{N+1}^*) + k_1 \mathcal{M} \mathbf{1}_N \otimes \tilde{h}_{N+1}^* = 0 \quad (56)$$

となり、特殊解が非斉次方程式を満たすことが確認できた。

得られた特殊解の物理的意味を考える。元の変数変換を思い出すと、 $\tilde{h}_i(t) = r_i(t) - l_i d_i(t)$ であった。ここで、 r_i はエージェント i の位置、 d_i は目標相対位置、 l_i はリーダーフラグである。

特殊解 $\tilde{h}_A^* = \mathbf{1}_N \otimes \tilde{h}_{N+1}^*$ は、すべてのエージェント ($i = 1, 2, \dots, N$) について $\tilde{h}_i^* = \tilde{h}_{N+1}^*$ となることを意味する。これを元の変数に戻すと、

$$r_i^* - l_i d_i^* = \tilde{h}_{N+1}^* \quad (57)$$

$$r_i^* = \tilde{h}_{N+1}^* + l_i d_i^* \quad (58)$$

ここで、 $\tilde{h}_{N+1}^* = r_{N+1}^*$ はリーダーの目標位置である。フォロワー ($l_i = 1$) については、

$$r_i^* = r_{N+1}^* + d_i^* \quad (59)$$

となり、フォロワーはリーダーの目標位置から目標相対位置 d_i^* だけ離れた位置に収束することがわかる。つまり、フォロワーはリーダーから見た目標相対位置 d_i^* を維持しフォーメーションが形成される。

また、 $\tilde{v}_A^* = 0$ は、定常状態ではすべてのフォロワーの速度がゼロになることを意味し、これはリーダーの目標速度がゼロであることと一致する。

以上より、制御則 (2) を適用したシステムにおいて、 $k_1 > 0$ かつ $k_2 > 0$ を満たすよう制御ゲインを選択すれば、リーダーは目標位置 r_{N+1} に収束し、フォロワーはリーダーから見た目標相対位置 d_i を維持しながらリーダーに追従することが示され、定理 1 が証明された。

4.5 実機用制御則のシミュレーション

この項では、制御則 (2) によるリーダー故障への対応の動作確認を行うためのシミュレーションを行う。概ねモーションキャプチャスペース内と同規模の移動距離となるよう、シミュレーションを実施した。ハードオーバー故障が $t \geq 3$ の時に発生するものとし、提案制御則の有無での結果の比較を実施した。

Table 1: Simulation Parameters		
Parameter	Symbol	Value
Formation control position gain	k_1	40
Formation control velocity gain	k_2	10
Distributed observer gain	m	5
Threshold of fault detection	τ	0.2
Reliability recovery rate	α	0.1
Sensitivity of fault	β	0.2
Failure signal (hard-over condition)	$f_{\max}$	$[-0.2 \ 0.2]^T$

シミュレーションの詳細な条件を Table 1 に示す。

提案制御則有の結果を Fig. 9 に，提案制御則無の結果を Fig. 10 に示す（動画も上映する）。図は左から順に，移動軌跡，目標相対位置からの偏差を示したコンセンサスエラー，残差を示す。

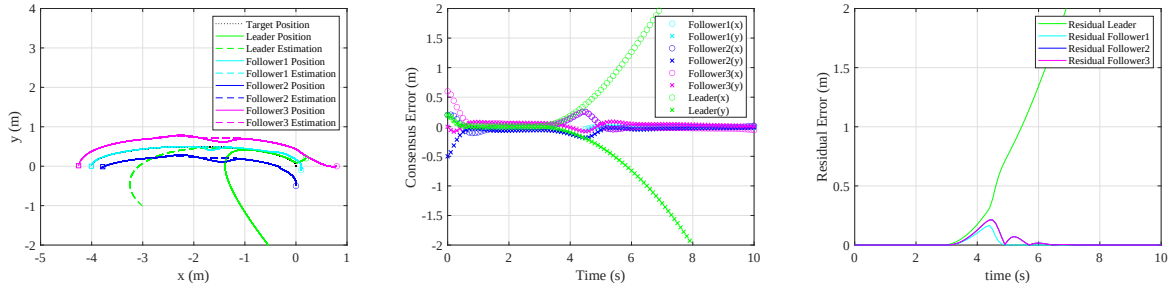


Fig. 9: Second-Order Formation under Leader Failure using the Proposal Method (Left: Trajectory, Center: Consensus Error, Right: Residual Errorsensus error, Right: Residual error)

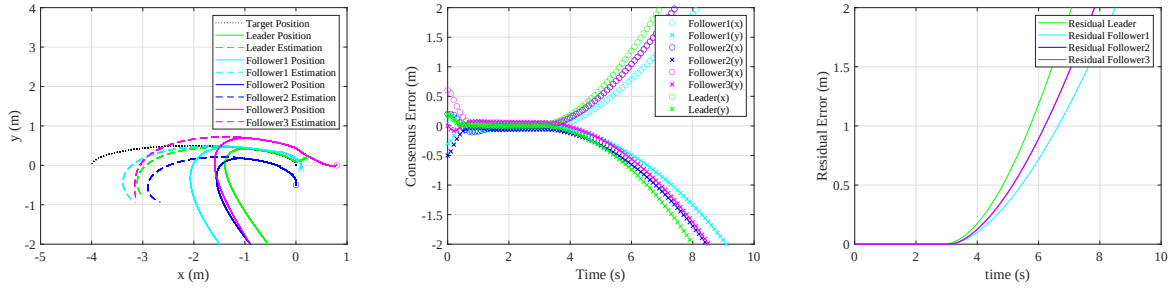


Fig. 10: Formation under disturbances of 2nd-order dynamics (left: Trajectory, right: Residual error)

ここまでの結果により，実機実装用の 2 次系の制御則に関して，理論，シミュレーション双方の観点で妥当性を確認することができた。

5 理論的發展に関する検討

2025 年度の研究進捗に向け，各エージェントが得られる局所的な情報に基づく分散的故障検知，故障エージェントからの衝突回避を実現するための基本コンセプトの検討を行った。

5.1 分散的な故障検知

4月ゼミで紹介した論文 [12] は、受信した情報を自身が既に持っている情報と、他エージェントから得た情報と照合し、矛盾を検出する異常エージェントの隔離アルゴリズム開発に軸足を置いていた。そのため、状態推定そのものについて着目した論文についてサーベイを行ったところ、ビザンチン攻撃環境下において、局所的な状態推定を統合することで全体としてのセキュア状態推定を可能とする事を目的とした、“Distributed Byzantine Resilient Observer (DBRO)” に関する研究 [21, 22] はリーダーフォロワー構造を前提としていること、その上でレジリエントなオブザーバ設計を実現することを目指している点から自身との研究の親和性が高いと考えており、[22] の再現を通じて課題抽出を行う。

5.1.1 Distributed Byzantine Resilient Observer [22] の概要

DBRO は、Byzantine 攻撃に対して耐性を持つ分散型状態推定手法であり、高次マルチエージェントシステムにおいて、リーダーの状態を攻撃存在環境下で正確に推定することを目的としている。主要な貢献点としては、高次系も含めて理論的な証明を行ったこと、DBRO の収束条件を、“strongly $(2f + 1)$ -robust” (各部分集合が、自身が隣接している攻撃エージェント数 f に対し、 $2f + 1$ 個の経路が存在すること) に緩和したことがあげられる。

5.1.2 問題設定

今回対象とする問題は、有向グラフの高次マルチエージェントシステムにおいて、Byzantine 攻撃 (f 機のフォロワーによる攻撃) が存在する環境下でフォロワーがリーダーの状態を正確に推定することである。まず、リーダーのダイナミクスは高次積分器システムとして表現される。

$$\dot{x}_{0,1} = x_{0,2}, \dot{x}_{0,2} = x_{0,3}, \dots, \dot{x}_{0,n} = u_0(x_0, t)$$

ここで、 $x_0 = [x_{0,1}, x_{0,2}, \dots, x_{0,n}]^T \in \mathbb{R}^n$ はリーダーの状態、 $u_0(x_0, t) \in \mathbb{R}$ はリーダーの入力、 $\|u_0(x_0, t)\|_\infty \leq \bar{u}_0$ を満たす ($\bar{u}_0$ は既知のスカラーであり、これによりリーダーの入力が有界となる)。

通信構造は有向グラフ $\mathcal{G} = (\mathcal{V}, \mathcal{E}, A)$ として表現される。ノード集合は $\mathcal{V} = \{0, 1, 2, \dots, N\}$ (0 はリーダー、 1 から N はフォロワー) で表される (4章までの定義と異なっているため注意されたい)。エッジ集合は、 $\mathcal{E} \subset \mathcal{V} \times \mathcal{V}$ で、隣接行列: $A = [a_{ij}]$ で異常エージェント集合 $\mathcal{B}$ 、正常エージェントの集合は $\mathcal{H}$ で表し、各フォロワーが情報を受け取る異常な隣接ノードは最大でも f 個とし (これを、 f -local という)、グラフ構造全体としては strongly $(2f + 1)$ -robust を満たすものとする。

5.1.3 DBRO アルゴリズム

DBRO アルゴリズムのコアアイデアは、符号関数を用いた状態推定である。

まず、拡張隣接集合 $\mathcal{M}$ を次式のように定義する。拡張隣接集合 $\mathcal{M}_i$ は、各フォロワーエージェント i がリーダーの状態を推定する際に参照する情報の集合を定義するものであり、特にリーダーに直接接続されているフォロワーとそうでないフォロワーとで扱いが異なる。

フォロワー i がリーダー j から接続されたフォロワーの集合 $\mathcal{F}_p$ に属する場合、その拡張隣接集合 $\mathcal{M}_i$ は、通常のエージェント間の隣接集合 $\mathcal{N}_i$ に加え、リーダーからの情報をコピーした仮想的な f 個の情報源 $I[N + 1, N + f]$ を含む。具体的には以下のように定義される。

$$\mathcal{M}_i = \mathcal{N}_i \cup I[N + 1, N + f], \quad \text{if } i \in \mathcal{F}_p \quad (60)$$

この拡張により、リーダーからの情報を自身が隣接している異常エージェント数 f を上回る $(f+1)$ 機分の重みとして扱うことになる。一方、フォロワー i がリーダーと隣接していないフォロワーの集合 $\mathcal{F}_{np}$ に属する場合、拡張隣接集合は通常のエージェント間の隣接集合と同じとなり、次式で表される。

$$\mathcal{M}_i = \mathcal{N}_i, \quad \text{if } i \in \mathcal{F}_{np} \quad (61)$$

この拡張隣接集合 $\mathcal{M}_i$ は、後述する DBRO の更新則における重要な項である ϕ_k^i の計算に用いられる。 ϕ_k^i は、フォロワー i の状態推定値と、 $\mathcal{M}_i$ に含まれる他のエージェント（または仮想情報源）の状態推定値との差の符号によって重み付けされた和として構成される。これにより、ビザンチン攻撃に頑健な状態推定を目指す。

状態推定アルゴリズム、特に DBRO の更新則において中心的な役割を果たすのが、次式で定義される ϕ_k^i である。

$$\phi_k^i = f \cdot a_{i0} \text{sgn}(\hat{x}_{0,k}^i - x_{0,k}) + \sum_{j \in \mathcal{N}_i} a_{ij} \text{sgn}(\hat{x}_{0,k}^i - \hat{x}_{0,k}^{j \rightarrow i}) \quad (62)$$

これは、拡張隣接集合 $\mathcal{M}_i$ を用いると以下のように書き換えられる。

$$\phi_k^i = \sum_{j \in \mathcal{M}_i} a_{ij} \text{sgn}(\hat{x}_{0,k}^i - \hat{x}_{0,k}^{j \rightarrow i})$$

ここで、 $\hat{x}_{0,k}^i$ はフォロワー i によるリーダーの k 次の状態の推定値、 $x_{0,k}$ はリーダーの実際の k 次の状態（ピン止めされたフォロワーのみがアクセス可能）、 $\hat{x}_{0,k}^{j \rightarrow i}$ はエージェント j から i へ送信される k 次の状態情報（ビザンチン攻撃により改ざんされている可能性がある）、 a_{ij} はエージェント間の隣接行列の要素である。

この ϕ_k^i は、符号情報に落とし込まれたフィードバック項であり、フォロワー i の推定値と隣接ノード（またはリーダー）の情報の差の向きのみを利用する。残差を符号関数で処理することにより、異常値、外れ値が極めて大きな値をとった場合でも、実質的に単純な一票として扱い、影響を受けなくすることができるという点が特徴的である。 $\hat{x}_{0,k}^{j \rightarrow i}$ はエージェント j からエージェント i に送信される $\hat{x}_{0,k}^j$ の情報であり、異常エージェントの場合は偽情報が送信される可能性がある。

ここで、 $\text{sgn}(\cdot)$ は符号関数であり、 x が正の場合は 1 を x が 0 の場合は 0 を、 x が負の場合は -1 を出力する（理論的には、このままでは符号関数が $x = 0$ において不連続であり証明が困難となることから、理論的な証明のために、符号関数集合として、 $\text{SGN}(\cdot)$ を定義し、 $\text{SGN}(0) = [-1, 1]$ とすることで非平滑部分の処理を行い、証明をスムーズにしている）。

次に DBRO の更新則について説明する。リーダーの状態の推定値は以下のダイナミクスに従って更新される。

$$\hat{x}_{0,k}^i = \hat{x}_{0,k+1}^i - \alpha_k \cdot \phi_k^i \quad \forall k \in [1, n-1] \quad (63)$$

$$\hat{x}_{0,n}^i = -\alpha_n \cdot \phi_n^i \quad (64)$$

ここで、 α_k ($k \in [1, n]$) はオブザーバゲインであり、最高次 (n) のゲイン α_n は、リーダーの入力上限に対して上回り $\alpha_n > \bar{u}_0$ を満たす必要がある。その他のゲイン α_k は $\alpha_k > 0$ であればよい。

DBRO アルゴリズムは、前述の通り、リーダーの入力は有界で $\|u_0(x_0, t)\|_\infty \leq \bar{u}_0$ 、攻撃数が f -local であり、グラフ構造が strongly $(2f+1)$ -robust であるという仮定を満たす際、攻撃環境下でもリーダー状態の推定誤差は、有限時間内に 0 に収束することが保証されることが示されている。

5.1.4 シミュレーション

ここから、論文を再現したシミュレーションについて説明する。再現条件として、状態は、一次元の位置、速度、加速度とし、加速度入力システムとして、リーダーへの入力は -0.5 から 0.5 の間で周期的に変化する設定とし、それを踏まえてオブザーバゲインはリーダーの最大入力値を上回るように $\alpha_k = 1$ ($k \in [1, n]$) とした。異常

エージェントは2号機とし、加速度固定値0.1を隣接エージェントに送信し続けるという条件でシミュレーションを行った。隣接行列 $\mathcal{A}_1$ は次式の通りである。

$$\mathcal{A}_1 = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

次に、強ロバスト性が満たされた際の DBRO 無, 有のシミュレーション結果をそれぞれ Fig. 11, Fig. 12 に示す。DBRO 無だと、異常エージェントの値に引っ張られる形で誤った推定値となっているが、有の場合だと、DBRO が機能したことにより正しくリーダーの状態推定ができている。

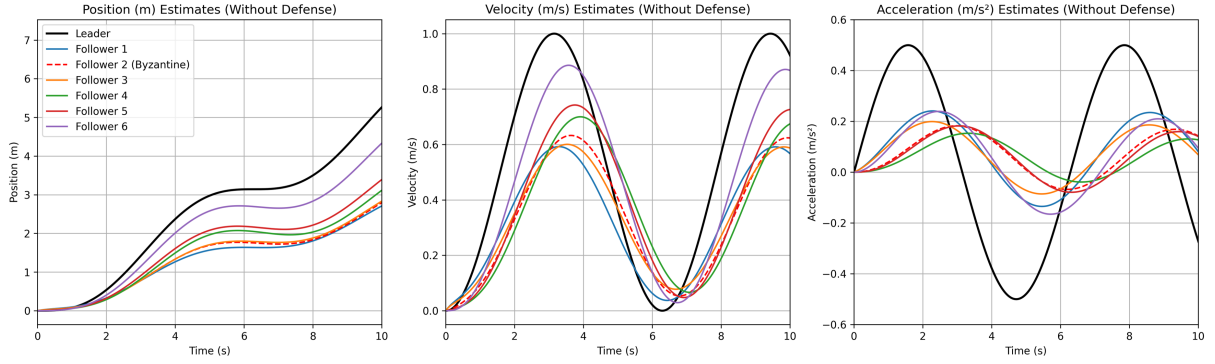


Fig. 11: State estimation without DBRO

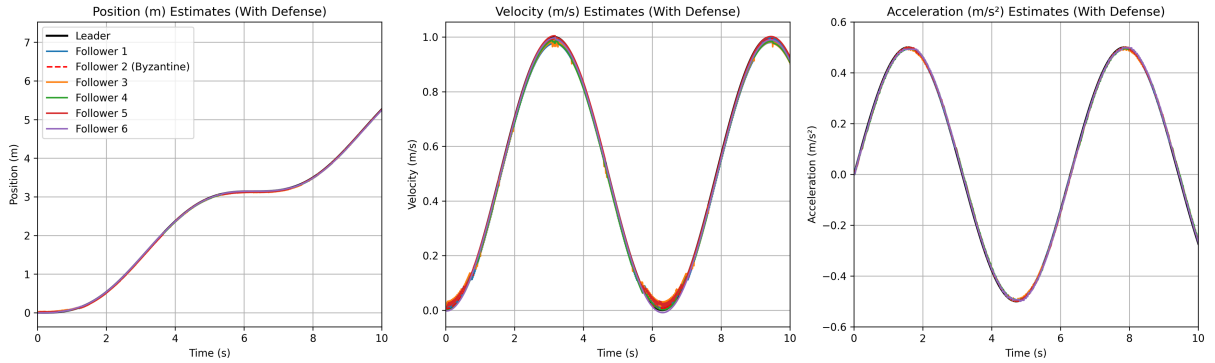


Fig. 12: State estimation with DBRO

強ロバストの条件を落とした場合についてシミュレーションを行った。隣接行列先ほどの条件の $\mathcal{A}_1$ と差分がある箇所を赤字で記載する。

$$\mathcal{A}_2 = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

結果を Fig. 13 に示す. リーダーから直接接続されている 1, 6 号機のみ正しく状態推定できているが, 他エージェント (3,4,5 号機) は誤った値 (加速度) を採用し, 誤った値に収束していることがわかる,

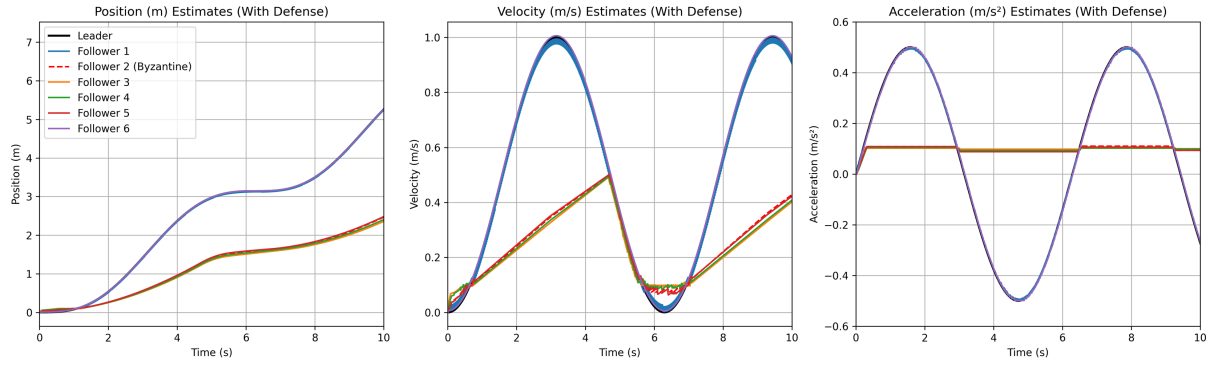


Fig. 13: State estimation under weak network structure

これらの結果により, DBRO が仮定のもとでは良好に動作することが確認できた.

今回再現した論文はリーダーの状態推定に着目していたところであるが, 自身の研究への取り込みにあたっては, 今回の方式を拡張するような形でリーダーフォロワーを包含した形での状態推定を行う定式化により自身の研究への取り込みを行うことや, 自身の研究の課題として考えられる, 非故障エージェントの信頼度が十分減少する前に極めて大きな故障信号が入った場合, 故障エージェントもあわせて大きな影響を受けるような点の課題解決に用いることができるといったアイデアを考えている.

6 まとめ・今後の方向性

今回, 2024 年度研究の実機検証の開始に向け, 機体特性の取得, 複数 UAV の同時制御の実装を行いつつ, 実機実装用の制御則の検討, 動作確認を行った. 加えて 2025 年度の研究方向性の検討のため, 関連論文のアルゴリズムの再現を行った. 直近は特に実機検証を最優先事項と捉えた上で以下の 3 点を着実に進めていきたい.

- 複数 UAV のモーションキャプチャによる位置フィードバックも交えた複数機のフォーメーション飛行を実現した上で 2024 年度研究の実機実証を行う.
- 分散的な故障検知及びリーダー交代に関する提案制御則の定式化及びシミュレーションを実施する.
- 故障エージェントからの衝突回避を可能とする方策について, 時変 CBF などの適用可能性 (基礎検討は Appendix に記載) について検討を行う.

参考文献

- [1] J. Cortes and M. Egerstedt, “Coordinated Control of Multi-Robot Systems: A Survey,” *SICE Journal of Control, Measurement, and System Integration*, Vol. 10, No. 6, pp. 495-503, 2017.
- [2] M.R. Davoodi, N. Meskin, and K. Khorasani, “Simultaneous Fault Detection and Control Design for a Network of Multi-Agent Systems,” *Automatica*, Vol. 66, pp. 185-194, 2016.
- [3] P. Yang, B. Ma, Y. Dong and J. Liu, “Adjustable Parameter-Based Distributed Fault Estimation Observer Design for Multiagent Systems With Directed Graphs,” *IEEE Transactions on Cybernetics*, Vol. 47, No. 2, pp. 306-314, 2017.
- [4] K. Zhang, B. Jiang and P. Si, “Fault-tolerant Consensus of Leader-Following Multi-Agent Systems Based on Distributed Fault Estimation Observer,” *International Journal of Control, Automation and Systems*, Vol. 16, No. 5, pp. 2354-2362, 2018.
- [5] Y. Wu, Z. Wang, S. Ding and H. Zhang, “Leader-Follower Consensus of Multi-Agent Systems in Directed Networks With Actuator Faults,” *Neurocomputing*, Vol. 275, No. 31, pp. 1177-1185, 2018.
- [6] H. Wang and J. Dong, “Robust Fault-Tolerant Formation Maneuver Control for Multiagent Systems With Mismatched Disturbances,” *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, Vol. 54, No. 6, pp. 3762-3773, 2024.
- [7] G. L. Mariottini, F. Morbidi, D. Prattichizzo, N. V. Valk, N. Michael, G. Pappas and K. Daniilidis, “Vision-Based Localization for Leader-Follower Formation Control,” *IEEE Transactions on Robotics*, Vol. 25, No. 6, pp. 1431-1438, 2009.
- [8] Z. Li, X. Liu, W. Ren and L. Xie, “Distributed Tracking Control for Linear Multiagent Systems With a Leader of Bounded Unknown Input,” *IEEE Transactions on Automatic Control*, Vol. 58, No. 2, pp. 518-523, 2013.
- [9] Y. Kuriki and T. Namerikawa, “Consensus of Formation Configuration Using Leader-Follower Structure,” *Journal of System Design and Dynamics*, Vol. 7, No. 3, pp. 254-264, 2013.
- [10] K. Kotani, Z. Guo, T. Namerikawa and Z. Qu, “Cooperative Transport Control by a Multicopter System,” *IET Control Theory and Applications*, Vol. 15, No. 6, pp. 861-876, 2021.
- [11] H. Rezaee, T. Parisini and M. M. Polycarpou, “Resiliency in Dynamic Leader-Follower Multiagent Systems,” *Automatica*, Vol. 125, 109384, 2021.
- [12] J. Li, Y. Hua, J. Yu, X. Dong, Z. Feng and Z. Ren, “Resilient Time-Varying Formation Tracking Control for General Linear Multiagent Systems With a Nonautonomous Leader and Adversarial Followers,” *IEEE Transactions on Cybernetics*, vol. 54, No. 10, pp. 6203-6216, 2024.
- [13] F. Li, T. Ding, M. Zhou, K. Hao and L. Chen, “An Affection-Based Dynamic Leader Selection Model for Formation Control in Multirobot Systems,” *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, Vol. 47, No. 7, pp. 1217-1228, 2017.
- [14] T. Murakami and T. Namerikawa, “Consensus Control for Resilient Multi-Agent Systems Considering Leader Failure,” *SICE International Symposium on Control Systems 2025*, pp. 38-42, Osaka, Japan, 2025.

- [15] T. Murakami and T. Namerikawa, “Resilient Formation Control in Multi-Agent Systems Considering Leader Failure,” *SICE Journal of Control, Measurement, and System Integration*, Vol.18, No.1, 2510766, 2025.
- [16] J. Boskovic and R. K. Mehra, “Failure Detection, Identification and Reconfiguration in Flight Control,” *Fault Diagnosis and Fault Tolerance for Mechatronic Systems: Recent Advances*, Chapter 1, pp. 129-167, Springer, 2003.
- [17] S.X. Ding, “Model-Based Fault Diagnosis Techniques: Design Schemes, Algorithms, and Tools,” Springer, 2013.
- [18] Y. Kuriki and T. Namerikawa, “Formation Control of UAVs with a Fourth-Order Flight Dynamics,” *Journal of Control, Measurement, and System Integration*, Vol. 7, pp. 74-81, 2014.
- [19] 小谷健人, 滑川徹, “マルチ Quad-rotor による協調搬送システムに向けた RISE による外乱補償,” 電気学会論文誌 C, Vol.139, No.4, pp. 341-349, 2019.
- [20] J. Li, Y. Hua, J. Yu, X. Dong, Z. Feng and Z. Ren, “Resilient Time-Varying Formation Tracking Control for General Linear Multiagent Systems With a Nonautonomous Leader and Adversarial Followers,” *IEEE Transactions on Cybernetics*, vol. 54, No. 10, pp. 6203-6216, 2024.
- [21] A. Mitra and S. Sunram, “Byzantine-Resilient Distributed Observers for LTI Systems,” *Automatica*, vol. 108, 108487, 2019.
- [22] X. Gong, X. Li and Z. Shu, “Distributed Byzantine-Resilient Observer for High-Order Integrator Multiagent Systems on Directed Graphs: An Edge-Based Approach,” *IEEE Transactions on Automatic control*, vol. 69, No.5 2024.
- [23] A.D. Ames, X. Xu, J.W. Grizzle and P. Tabuada, “Control Barrier Function Based Quadratic Programs for Safety Critical Systems,” *IEEE Transactions on Automatic Control*, Vol. 62, pp.3861-3876, 2017.
- [24] M. Egerstedt, “Robot Ecology: Constraint-Based Design for Long-Duration Autonomy,” Chapter 5.2, Princeton, NJ, USA: Princeton University Press, 2021.
- [25] 五十嵐基, 高井麻希, 中村文一, “制御バリア関数を用いた時変の障害物回避のためのヒューマンアシスト制御,” 計測自動制御学会論文集, Vol.56, No.9, pp.432-441, 2020.
- [26] 手塚一成, 松浦隼, 中村文一, “厳密微分機を用いた移動体障害物回避制御,” 計測自動制御学会論文集, Vol.57, No.6, pp.276-284, 2021.

Appendix A 行列 $\mathcal{M}$ の正定性について

行列 $\mathcal{M}$ の正定性は二次形式を通じて評価できる．仮定 1,2 が満たされる場合，行列 $\mathcal{M}$ は，リーダー行の要素が 0 に設定されている以外は実質的に対称行列として扱うことができる．この対称性を利用すると，二次形式は次式のように表現できる．

$$x^\top \mathcal{M} x = \frac{1}{2} \sum_{i=1, j=1}^N w_{ij} (x_i - x_j)^2 + \sum_{i=1}^N (1 - l_i) x_i^2 \quad (65)$$

ここで，式 (65) の第 1 項は， $x_i = x_j \neq 0$ の場合に 0 となる．一方，式 (65) の第 3 項は，リーダーが 1 機存在するため常に正である．したがって，二次形式はすべての $x_i \neq 0$ に対して正となる．二次形式の定義から，行列 $\mathcal{M}$ は正定であることがわかる．

Appendix B 時変 CBF

時変 CBF の概要

制御バリア関数 (Control Barrier Function: CBF) [23] は，制御入力によってシステムの状態が所定の安全集合内に留まることを保証するための手法である．通常の CBF は，状態空間において不変な安全集合を対象として定義される．一方，移動障害物回避や動的環境においては，安全集合そのものが時間的に変化するため，CBF を時変に拡張した時変制御バリア関数 (Time-varying CBF) [24] の導入により解決可能と考えられる．なお，Time-varying CBF については，Ames らの研究を引用する形で Magnus が本としてまとめた [24] もの他，国内では中村らによる研究 [25, 26] が存在しており，定式化や考え方について相違が見られるが，本研究においては Magnus のものに準拠したものとする．

システムの状態を $x \in \mathbb{R}^n$ ，制御入力を $u \in \mathbb{R}^m$ とし，次式のアフィンシステムを考える．

$$\dot{x} = f(x) + g(x)u, \quad (66)$$

ここで， $f(x)$ および $g(x)$ はそれぞれ連続で局所 Lipschitz 連続であるとする．安全集合 $C(t)$ は時変バリア関数 $h(x, t)$ を用いて以下のように定義される．

$$C(t) = \{x \in \mathbb{R}^n \mid h(x, t) \geq 0\} \quad (67)$$

ただし， $h: \mathbb{R}^n \times \mathbb{R}_+ \rightarrow \mathbb{R}$ は状態 x と時刻 t に明示的に依存する連続微分可能な関数である．

時変 CBF の定義 関数 $h(x, t)$ が次の条件を満たすとき， $h(x, t)$ を時変制御バリア関数と呼ぶ

$$\sup_{u \in \mathbb{R}^m} \left\{ \frac{\partial h}{\partial t}(x, t) + L_f h(x, t) + L_g h(x, t)u + \alpha(h(x, t)) \right\} \geq 0 \quad (68)$$

ここで， α は拡張クラス $\mathcal{K}_\infty$ 関数（例えば $\alpha(s) = \gamma s$, $\gamma > 0$ ）であり， $L_f h(x, t) = \frac{\partial h}{\partial x} f(x)$, $L_g h(x, t) = \frac{\partial h}{\partial x} g(x)$ はそれぞれ Lie 導関数を表す．

通常の CBF に比べ，時変 CBF には安全領域が時間的に変化するという特徴がある．上式に現れる $\frac{\partial h}{\partial t}(x, t)$ は，「安全集合の境界が時間経過とともにどのように動くか」を定量的に表している．つまり，状態が停止したままであっても，安全集合自体が移動することで安全性が損なわれる可能性を考慮する項である．

制御則を設計する際には，QP 問題などに落とし込み，制御入力 u を適切に選択して，この時変安全領域から逸脱しない条件を満たすことにより安全が保証されることとなる．

例えば，単純な線形一次系において，時刻 t における障害物の位置を $p_o(t)$ とすると，安全距離 d_{safe} を確保するためのバリア関数は次のようになる．

$$h(x, t) = \|x - p_o(t)\|^2 - d_{\text{safe}}^2 \quad (69)$$

このとき、 $\frac{\partial h}{\partial t}(x, t)$ は以下のように計算される (障害物が近接してくる速さに対応する).

$$\frac{\partial h}{\partial t}(x, t) = -2(x - p_o(t))^\top \dot{p}_o(t). \quad (70)$$

この場合は,

$$\sup_{u \in \mathbb{R}^m} \{-2(x - p_o(t))^\top \dot{p}_o(t) + 2(x - p_o(t))^\top u + \alpha(h(x, t))\} \geq 0 \quad (71)$$

の条件を満たすよう u を決定する, もしくは QP の制約式として扱うことで, 動的な障害物からの衝突回避を保証できる.

自身の研究への適用検討

ただし, 前述の時変 CBF を適用するためには, 対象の速度が完全に既知であるか, 既知でない場合も厳密微分器を設計することにより, 速度推定値を誤差の upper-bound を明らかにした上で推定可能とする必要があった [26].

2024 年度時点で未考慮だった故障エージェントからの衝突回避を可能とするための課題としては, 「故障エージェントが移動体であること」, 「故障エージェントの挙動の情報が直接入手困難」という 2 点が挙げられる. この 2 点のうち前者は時変 CBF で解決が見込まれるが, 後者については別途解決の方策について検討を行う必要がある. 一案として, 発生故障が四種類のいずれかにあてはまるものと仮定すると, 故障種別の識別後はその後の挙動が明らかになる (もしくは故障信号の upper-bound を明らかにすることができる) ため, その情報を時変で織り込んだバリア関数が設計可能となると考えられる. そのために必要となる, 故障類型を特定するための仕組みを検討・構築するような方向性が考えられる.